Teste 5 da PoC: FedAvg em claro × SMPC × CKKS (mesmo Autoencoder)

Objetivo: medir o custo e o impacto na qualidade de cada mecanismo de agregação segura, comparando **FedAvg em claro × SMPC** e **FedAvg em claro × CKKS** com o **mesmo modelo Autoencoder** (arquitetura do Teste 2).

Controle de variáveis: as três execuções usam o mesmo dataset, o mesmo split 60/20/20 estratificado, o mesmo scaler (ajustado só no treino), a mesma partição entre clientes, os mesmos pesos iniciais e as mesmas sementes de treino local. **A única diferença entre elas é o mecanismo de agregação.**

Mecanismo	O que o servidor vê	Como a média é f
FedAvg em claro	os pesos de cada cliente	soma ponderada em float64 (referência exata)
SMPC	apenas somas parciais mascaradas	secret sharing aditivo em campo finito (primo c
CKKS	apenas cifrados (contexto **sem** chave secreta)	soma homomórfica no TenSEAL; só os clientes

Em todos os casos o FedAvg é **ponderado** (peso de cada cliente = n_k / N), aplicado pelo próprio cliente antes de proteger seus pesos.

Métricas: ROC-AUC, PR-AUC, Precisão, Recall, F1 e Acurácia no teste (limiar escolhido na validação); erro numérico da agregação em relação ao FedAvg exato em cada rodada; concordância das predições com o modelo em claro; tempo de proteção, agregação e revelação; e bytes enviados por cliente por rodada.

```
!pip install -q tenseal
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.2/5.2 MB 35.5 MB/s eta 0:00:00
```

```
# -*- coding: utf-8 -*-
"""
Teste 5 da PoC: Comparação de Agregação Segura em Aprendizado Federado
FedAvg em claro × SMPC  e  FedAvg em claro × CKKS (TenSEAL)
Modelo: Autoencoder Tabular (PyTorch), o mesmo do Teste 2
"""

import os
import copy
import time
import random

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import RobustScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             roc_auc_score, average_precision_score, precisi
                             roc_curve)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torch.nn.utils import parameters_to_vector, vector_to_parameters

try:
    import tenseal as ts
except ImportError:
    raise ImportError("TenSEAL não está instalado. Execute: pip install tens

# ======================================================================
# --- 0. CONFIGURAÇÃO DO EXPERIMENTO ---
# ======================================================================
SEED = 42
NUM_CLIENTES = 3
NUM_RODADAS = 5          # Rodadas de comunicação do FL
EPOCAS_LOCAIS = 2        # Épocas de treino local por rodada
BATCH_SIZE = 256
LR = 1e-3
WEIGHT_DECAY = 1e-5

# Bases a executar (a que não for encontrada é pulada com aviso)
DATASETS = ['creditcard', 'paysim']
PAYSIM_AMOSTRA = 1_000_000   # Amostra estratificada da PaySim (None = base

CAMINHOS = {
    'creditcard': [
        '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/creditcard/cred
        'base_de_dados/creditcard.zip',        # Executando de dentro de PoC
        'PoC/base_de_dados/creditcard.zip',    # Executando da raiz do repos
    ],
    'paysim': [
        '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/paysim.c
    ],
}

# SMPC: secret sharing aditivo em Z_p
SMPC_MODULO = 2**61 - 1        # Primo de Mersenne: cabe em int64 mesmo soma
SMPC_BITS_PRECISAO = 24        # Ponto fixo: resolução de 2^-24 (mesmo valor
SMPC_ESCALA = 2 ** SMPC_BITS_PRECISAO

# CKKS (TenSEAL): mesmos parâmetros do Teste 3
CKKS_POLY_MOD = 8192
CKKS_COEFF_MOD = [60, 40, 40, 60]
CKKS_ESCALA_BITS = 40


def seed_everything(seed=SEED):
```

```
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False


seed_everything(SEED)
sns.set_style("whitegrid")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Dispositivo PyTorch: {device}")


def sincronizar():
    """Garante que o cronômetro só pare depois que a GPU terminar (CUDA é as
    if device.type == 'cuda':
        torch.cuda.synchronize()


try:
    from google.colab import drive
    drive.mount('/content/drive')
except ImportError:
    print("Fora do Google Colab: usando apenas caminhos locais.")
```

```
Dispositivo PyTorch: cuda
Mounted at /content/drive
```

```
# ======================================================================
# --- 1. CARREGAMENTO E PREPARAÇÃO DOS DADOS ---
# ======================================================================

def ler_base(nome):
    for caminho in CAMINHOS[nome]:
        if os.path.exists(caminho):
            print(f"Carregando '{caminho}'...")
            return pd.read_csv(caminho)
    return None


def preparar_creditcard(df):
    # Time é um contador monotônico: ruído para modelos baseados em distânci
    df = df.drop(columns=['Time'], errors='ignore').dropna(subset=['Class'])
    df['amount_log'] = np.log1p(df['Amount'].clip(lower=0))
    df = df.drop(columns=['Amount'])
    return df, 'Class'


def preparar_paysim(df):
    # Fraudes na PaySim ocorrem apenas em TRANSFER e CASH_OUT (EDA do Teste
    df = df[df['type'].isin(['TRANSFER', 'CASH_OUT'])].copy()
    df = df.drop(columns=[c for c in ['nameOrig', 'nameDest', 'isFlaggedFrau
```

```
        df = df.dropna(subset=['isFraud'])
        if PAYSIM_AMOSTRA and len(df) > PAYSIM_AMOSTRA:
            print(f"Amostragem estratificada de {PAYSIM_AMOSTRA:,} linhas (prese
            df, _ = train_test_split(df, train_size=PAYSIM_AMOSTRA, stratify=df[
        df['amount_log'] = np.log1p(df['amount'].clip(lower=0))
        df['errorBalanceOrig'] = df['newbalanceOrig'] + df['amount'] - df['oldba
        df['errorBalanceDest'] = df['oldbalanceDest'] + df['amount'] - df['newba
        df = df.drop(columns=['amount'])
        df = pd.get_dummies(df, columns=['type'], drop_first=True, dtype=float)
        return df, 'isFraud'


    PREPARADORES = {'creditcard': preparar_creditcard, 'paysim': preparar_paysim


    def carregar_dados(nome):
        df = ler_base(nome)
        if df is None:
            return None
        df, alvo = PREPARADORES[nome](df)
        for col in df.select_dtypes(include=np.number).columns:
            if df[col].isnull().any():
                df[col] = df[col].fillna(df[col].median())

        X = df.drop(columns=[alvo]).values.astype(np.float32)
        y = df[alvo].values.astype(np.int64)

        # Split 60/20/20 estratificado ANTES de escalar (sem data leakage)
        X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, r
        X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_s

        scaler = RobustScaler()
        X_train = scaler.fit_transform(X_train).astype(np.float32)
        X_val = scaler.transform(X_val).astype(np.float32)
        X_test = scaler.transform(X_test).astype(np.float32)

        # Autoencoder treina só com transações normais; cada cliente recebe uma
        X_train_normal = X_train[y_train == 0]
        clientes_X = np.array_split(X_train_normal, NUM_CLIENTES)
        n_total = sum(len(c) for c in clientes_X)
        pesos_fedavg = [len(c) / n_total for c in clientes_X]

        print(f"Treino: {X_train.shape} | Validação: {X_val.shape} | Teste: {X_t
              f"| Fraudes no teste: {int(y_test.sum())}")
        print(f"Amostras normais por cliente: {[len(c) for c in clientes_X]}")
        return dict(nome=nome, X_val=X_val, y_val=y_val, X_test=X_test, y_test=y
                    clientes_X=clientes_X, pesos_fedavg=pesos_fedavg, input_dim=
```

```
    # ================================================================
    # --- 2. MODELO: AUTOENCODER TABULAR (MESMA ARQUITETURA DO TESTE 2) ---
    # ================================================================

    class TabularAutoencoder(nn.Module):
        def __init__(self, input_dim):
```

```
            super(TabularAutoencoder, self).__init__()
            self.encoder = nn.Sequential(
                nn.Linear(input_dim, 24),
                nn.LeakyReLU(0.01),
                nn.Dropout(0.1),
                nn.Linear(24, 12),
                nn.LeakyReLU(0.01),
                nn.Linear(12, 6)  # Espaço latente (gargalo)
            )
            self.decoder = nn.Sequential(
                nn.Linear(6, 12),
                nn.LeakyReLU(0.01),
                nn.Linear(12, 24),
                nn.LeakyReLU(0.01),
                nn.Linear(24, input_dim)
            )

        def forward(self, x):
            return self.decoder(self.encoder(x))


    def modelo_para_vetor(modelo):
        return parameters_to_vector(modelo.parameters()).detach().cpu().to(torch


    def vetor_para_modelo(vetor, modelo):
        vector_to_parameters(vetor.to(dtype=torch.float32, device=device), model


    def treinar_local(estado_global, dados_X, input_dim, rodada, cliente_id):
        """Treino local idêntico para todos os mecanismos: mesma semente por (ro
        semente = SEED + 1000 * rodada + cliente_id
        torch.manual_seed(semente)  # Controla o Dropout
        modelo = TabularAutoencoder(input_dim).to(device)
        modelo.load_state_dict(estado_global)
        modelo.train()
        optimizer = optim.Adam(modelo.parameters(), lr=LR, weight_decay=WEIGHT_D
        criterion = nn.MSELoss()

        gerador = torch.Generator().manual_seed(semente)  # Controla o shuffle
        loader = DataLoader(TensorDataset(torch.from_numpy(dados_X)), batch_size
                            shuffle=True, generator=gerador)
        for _ in range(EPOCAS_LOCAIS):
            for (lote,) in loader:
                lote = lote.to(device, non_blocking=True)
                optimizer.zero_grad()
                loss = criterion(modelo(lote), lote)
                loss.backward()
                optimizer.step()
        return modelo_para_vetor(modelo)


    def erros_reconstrucao(modelo, X, tamanho_lote=65536):
        modelo.eval()
        saida = []
```

```
        with torch.no_grad():
            for i in range(0, len(X), tamanho_lote):
                lote = torch.from_numpy(X[i:i + tamanho_lote]).to(device)
                saida.append(torch.mean((lote - modelo(lote)) ** 2, dim=1).cpu()
        return np.concatenate(saida)
```

```
# ====================================================================
# --- 3. MECANISMOS DE AGREGAÇÃO ---
# Todos recebem os mesmos vetores de pesos locais e os pesos do FedAvg,
# e devolvem (vetor agregado, custos da rodada).
# ====================================================================

class AgregadorClaro:
    """FedAvg em claro: o servidor recebe os pesos de cada cliente sem prote
    nome = 'FedAvg em claro'

    def __init__(self):
        self.tempo_setup = 0.0

    def agregar(self, vetores, pesos):
        t0 = time.perf_counter()
        pacotes = [v.to(torch.float32) for v in vetores]  # Pesos brutos do
        t_protecao = (time.perf_counter() - t0) / len(vetores)

        t0 = time.perf_counter()
        agregado = sum(p.to(torch.float64) * a for p, a in zip(pacotes, peso
        t_agregacao = time.perf_counter() - t0

        bytes_cliente = pacotes[0].numel() * pacotes[0].element_size()
        return agregado, dict(t_protecao=t_protecao, t_agregacao=t_agregacao
                              bytes_cliente=bytes_cliente)


class AgregadorSMPC:
    """
    Secret sharing aditivo em Z_p com 3 nós (os próprios clientes).
    1. Cada cliente codifica seus pesos em ponto fixo e os divide em N share
       cuja soma (mod p) é o valor original; o share j vai para o nó j.
    2. Cada nó soma os shares recebidos (mod p): vê apenas ruído uniforme.
    3. O servidor soma as N somas parciais e obtém só o agregado, nunca um p
    """
    nome = 'SMPC'

    def __init__(self):
        self.tempo_setup = 0.0
        # Gerador próprio: não consome o RNG global, então o treino local fi
        # Numa implantação real os shares devem vir de um CSPRNG (ex.: secre
        self.gerador = torch.Generator().manual_seed(SEED + 777)

    @staticmethod
    def codificar(v):
        return torch.remainder(torch.round(v * SMPC_ESCALA).to(torch.int64),

    @staticmethod
```

```
        def decodificar(v_int):
            v_int = torch.where(v_int > SMPC_MODULO // 2, v_int - SMPC_MODULO, v
            return v_int.to(torch.float64) / SMPC_ESCALA

        def gerar_shares(self, v_int, n):
            shares = [torch.randint(0, SMPC_MODULO, v_int.shape, dtype=torch.int
                      for _ in range(n - 1)]
            ultimo = v_int.clone()
            for s in shares:
                ultimo = torch.remainder(ultimo - s, SMPC_MODULO)
            return shares + [ultimo]

        def agregar(self, vetores, pesos):
            n = len(vetores)
            caixa_entrada = [[] for _ in range(n)]  # caixa_entrada[j]: shares r

            # Clientes: codificam e distribuem os shares
            t0 = time.perf_counter()
            for v, a in zip(vetores, pesos):
                for j, share in enumerate(self.gerar_shares(self.codificar(v * a
                    caixa_entrada[j].append(share)
            t_protecao = (time.perf_counter() - t0) / n

            # Nós: soma parcial cega (mod p); Servidor: soma das parciais
            t0 = time.perf_counter()
            somas_parciais = []
            for j in range(n):
                soma = torch.zeros_like(caixa_entrada[j][0])
                for share in caixa_entrada[j]:
                    soma = torch.remainder(soma + share, SMPC_MODULO)
                somas_parciais.append(soma)
            total = torch.zeros_like(somas_parciais[0])
            for parcial in somas_parciais:
                total = torch.remainder(total + parcial, SMPC_MODULO)
            t_agregacao = time.perf_counter() - t0

            # Revelação: volta de Z_p para float
            t0 = time.perf_counter()
            agregado = self.decodificar(total)
            t_revelacao = time.perf_counter() - t0

            # Cada cliente envia (n-1) shares aos outros nós + 1 soma parcial ao
            bytes_vetor = total.numel() * total.element_size()
            return agregado, dict(t_protecao=t_protecao, t_agregacao=t_agregacao
                                  bytes_cliente=n * bytes_vetor)


    class AgregadorCKKS:
        """
        Criptografia homomórfica CKKS (TenSEAL).
        Os clientes compartilham o contexto com chave secreta; o servidor recebe
        público (sem chave secreta) e soma os cifrados sem conseguir decifrá-los
        """
        nome = 'CKKS'
```

```
        def __init__(self):
            t0 = time.perf_counter()
            self.ctx_cliente = ts.context(ts.SCHEME_TYPE.CKKS, poly_modulus_degr
                                          coeff_mod_bit_sizes=CKKS_COEFF_MOD)
            self.ctx_cliente.global_scale = 2 ** CKKS_ESCALA_BITS
            self.ctx_servidor = ts.context_from(self.ctx_cliente.serialize(save_
            assert not self.ctx_servidor.has_secret_key(), "O servidor não pode
            self.slots = CKKS_POLY_MOD // 2
            self.tempo_setup = time.perf_counter() - t0

        def agregar(self, vetores, pesos):
            n = len(vetores)

            # Clientes: cifram (em blocos de 'slots' valores) e serializam para
            t0 = time.perf_counter()
            pacotes = []
            for v, a in zip(vetores, pesos):
                valores = (v * a).tolist()
                blocos = [valores[i:i + self.slots] for i in range(0, len(valore
                pacotes.append([ts.ckks_vector(self.ctx_cliente, b).serialize()
            t_protecao = (time.perf_counter() - t0) / n
            bytes_cliente = sum(len(b) for b in pacotes[0])

            # Servidor: desserializa com o contexto PÚBLICO e soma homomorficame
            t0 = time.perf_counter()
            soma = [ts.ckks_vector_from(self.ctx_servidor, b) for b in pacotes[0
            for pacote in pacotes[1:]:
                for i, b in enumerate(pacote):
                    soma[i] = soma[i] + ts.ckks_vector_from(self.ctx_servidor, b
            agregado_serializado = [c.serialize() for c in soma]
            t_agregacao = time.perf_counter() - t0

            # Clientes: decifram o agregado com a chave secreta
            t0 = time.perf_counter()
            valores = []
            for b in agregado_serializado:
                valores.extend(ts.ckks_vector_from(self.ctx_cliente, b).decrypt(
            t_revelacao = time.perf_counter() - t0

            return torch.tensor(valores, dtype=torch.float64), dict(
                t_protecao=t_protecao, t_agregacao=t_agregacao, t_revelacao=t_re
                bytes_cliente=bytes_cliente)
```

```
# ======================================================================
# --- 4. EXECUÇÃO DO FL E AVALIAÇÃO ---
# ======================================================================

def executar_fl(agregador, dados, estado_inicial):
    input_dim = dados['input_dim']
    modelo_global = TabularAutoencoder(input_dim).to(device)
    modelo_global.load_state_dict(estado_inicial)
    historico = []

    for rodada in range(NUM_RODADAS):
```

```
            estado = copy.deepcopy(modelo_global.state_dict())
            t0 = time.perf_counter()
            vetores = [treinar_local(estado, dados['clientes_X'][k], input_dim,
                       for k in range(NUM_CLIENTES)]
            sincronizar()
            t_treino = time.perf_counter() - t0

            agregado, custos = agregador.agregar(vetores, dados['pesos_fedavg'])

            # Referência: FedAvg exato (float64) dos MESMOS pesos locais
            referencia = sum(v * a for v, a in zip(vetores, dados['pesos_fedavg'
            diff = (agregado - referencia).abs()

            vetor_para_modelo(agregado, modelo_global)
            auc_val = roc_auc_score(dados['y_val'], erros_reconstrucao(modelo_gl

            historico.append(dict(rodada=rodada + 1, t_treino=t_treino, erro_max
                                  erro_rmse=diff.pow(2).mean().sqrt().item(), ro
            print(f"  Rodada {rodada + 1}/{NUM_RODADAS} | ROC-AUC val: {auc_val:
                  f"| erro máx. agregação: {diff.max().item():.2e} "
                  f"| agregação: {1000 * (custos['t_protecao'] + custos['t_agreg
        return modelo_global, pd.DataFrame(historico)


    def escolher_limiar(y_val, erros_val):
        """Limiar que maximiza o F1 na VALIDAÇÃO (nunca no teste)."""
        precisions, recalls, thresholds = precision_recall_curve(y_val, erros_va
        f1 = np.divide(2 * precisions * recalls, precisions + recalls,
                       out=np.zeros_like(precisions), where=(precisions + recall
        idx = np.argmax(f1[:-1])  # O último ponto da curva não tem limiar assoc
        return thresholds[idx]


    def avaliar(modelo, dados):
        limiar = escolher_limiar(dados['y_val'], erros_reconstrucao(modelo, dado

        sincronizar()
        t0 = time.perf_counter()
        scores = erros_reconstrucao(modelo, dados['X_test'])
        sincronizar()
        t_inferencia = time.perf_counter() - t0

        y_test = dados['y_test']
        y_pred = (scores >= limiar).astype(int)
        return dict(
            limiar=limiar, scores=scores, y_pred=y_pred,
            **{'ROC-AUC': roc_auc_score(y_test, scores),
               'PR-AUC': average_precision_score(y_test, scores),
               'Precisão': precision_score(y_test, y_pred, zero_division=0),
               'Recall': recall_score(y_test, y_pred, zero_division=0),
               'F1': f1_score(y_test, y_pred, zero_division=0),
               'Acurácia': accuracy_score(y_test, y_pred),
               'TPS inferência': len(y_test) / t_inferencia})
```

```
# ======================================================================
# --- 5. EXPERIMENTO: CLARO × SMPC E CLARO × CKKS ---
# ======================================================================

MECANISMOS = [AgregadorClaro, AgregadorSMPC, AgregadorCKKS]
METRICAS = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1', 'Acurácia']
CORES = {'FedAvg em claro': '#1f77b4', 'SMPC': '#ff7f0e', 'CKKS': '#2ca02c'}
resultados_gerais = {}

for nome_base in DATASETS:
    print("\n" + "=" * 78)
    print(f" BASE: {nome_base.upper()}")
    print("=" * 78)
    dados = carregar_dados(nome_base)
    if dados is None:
        print(f"Aviso: base '{nome_base}' não encontrada em {CAMINHOS[nome_b
        continue

    # Pesos iniciais idênticos para os três mecanismos
    seed_everything(SEED)
    estado_inicial = copy.deepcopy(TabularAutoencoder(dados['input_dim']).to
    n_params = sum(t.numel() for t in estado_inicial.values())
    print(f"Autoencoder: {dados['input_dim']} features de entrada, {n_params

    execucoes = {}
    for Mecanismo in MECANISMOS:
        agregador = Mecanismo()
        print(f"\n--- {agregador.nome} (setup: {agregador.tempo_setup:.2f}s)
        modelo, historico = executar_fl(agregador, dados, estado_inicial)
        execucoes[agregador.nome] = dict(modelo=modelo, historico=historico,
                                         avaliacao=avaliar(modelo, dados), t

    # ----- Tabela comparativa -----
    claro = execucoes['FedAvg em claro']
    linhas = []
    for nome_mec, ex in execucoes.items():
        av, hist = ex['avaliacao'], ex['historico']
        overhead_ms = 1000 * (hist['t_protecao'] + hist['t_agregacao'] + his
        linha = {'Mecanismo': nome_mec, **{m: av[m] for m in METRICAS},
                 'Erro máx. agregação': hist['erro_max'].max(),
                 'Δ máx. pesos finais vs claro': (ex['vetor'] - claro['vetor
                 'Concordância pred. vs claro (%)': 100 * np.mean(av['y_pred
                 'Correlação scores vs claro': np.corrcoef(av['scores'], cla
                 'Overhead agregação (ms/rodada)': overhead_ms,
                 'Upload por cliente/rodada (KB)': hist['bytes_cliente'].ilo
                 'Setup (s)': ex['tempo_setup'],
                 'Treino local (s/rodada)': hist['t_treino'].mean()}
        linhas.append(linha)
    tabela = pd.DataFrame(linhas).set_index('Mecanismo')
    resultados_gerais[nome_base] = dict(tabela=tabela, execucoes=execucoes)

    pd.set_option('display.width', 200)
    print("\n>> MÉTRICAS DE QUALIDADE NO TESTE (limiar escolhido na validaçã
    print(tabela[METRICAS].map(lambda x: f"{x:.4f}").to_string())
    print("\n>> FIDELIDADE E CUSTO DA AGREGAÇÃO SEGURA:")
```

```
print(tabela.drop(columns=METRICAS).to_string(float_format=lambda x: f"{

print("\n>> COMPARAÇÕES PAREADAS (Δ = mecanismo seguro − claro):")
for seguro in ['SMPC', 'CKKS']:
    t = tabela.loc[seguro]
    deltas = " | ".join(f"Δ{m}: {t[m] - tabela.loc['FedAvg em claro', m]
    fator = t['Overhead agregação (ms/rodada)'] / max(tabela.loc['FedAvg
    banda = t['Upload por cliente/rodada (KB)'] / tabela.loc['FedAvg em
    print(f"  Claro × {seguro}: {deltas} | concordância {t['Concordância
          f"| overhead {fator:,.0f}x | banda {banda:.1f}x")

# ----- Visualizações -----
fig, axes = plt.subplots(2, 3, figsize=(20, 11))
fig.suptitle(f"Teste 5: FedAvg em claro × SMPC × CKKS ({nome_base})", fo

# 1. Métricas de qualidade
ax = axes[0, 0]
tabela[METRICAS].T.plot.bar(ax=ax, color=[CORES[m] for m in tabela.index
ax.set_title('Métricas no teste')
ax.set_ylim(0, 1.2)
ax.legend(loc='upper center', ncol=3)

# 2. Convergência
ax = axes[0, 1]
for nome_mec, ex in execucoes.items():
    estilo = '-' if nome_mec == 'FedAvg em claro' else '--'
    ax.plot(ex['historico']['rodada'], ex['historico']['roc_auc_val'], e
            color=CORES[nome_mec], label=nome_mec)
ax.set_title('ROC-AUC na validação por rodada')
ax.set_xlabel('Rodada')
ax.legend()

# 3. Curvas ROC no teste
ax = axes[0, 2]
for nome_mec, ex in execucoes.items():
    fpr, tpr, _ = roc_curve(dados['y_test'], ex['avaliacao']['scores'])
    estilo = '-' if nome_mec == 'FedAvg em claro' else '--'
    ax.plot(fpr, tpr, estilo, color=CORES[nome_mec], label=f"{nome_mec}
ax.plot([0, 1], [0, 1], color='gray', linestyle=':')
ax.set_title('Curva ROC no teste')
ax.set_xlabel('FPR')
ax.set_ylabel('TPR')
ax.legend(loc='lower right')

# 4. Erro numérico da agregação
ax = axes[1, 0]
for nome_mec in ['SMPC', 'CKKS']:
    h = execucoes[nome_mec]['historico']
    ax.plot(h['rodada'], h['erro_max'], marker='o', color=CORES[nome_mec
    ax.plot(h['rodada'], h['erro_rmse'], marker='x', linestyle=':', colo
ax.set_yscale('log')
ax.set_title('Erro da agregação vs FedAvg exato')
ax.set_xlabel('Rodada')
ax.legend()
```

```
        # 5. Tempo de agregação por etapa
        ax = axes[1, 1]
        tempos = pd.DataFrame({nome_mec: 1000 * ex['historico'][['t_protecao', '
                               for nome_mec, ex in execucoes.items()}).T
        tempos.columns = ['Proteção (por cliente)', 'Agregação', 'Revelação']
        tempos.plot.bar(stacked=True, ax=ax, rot=0, color=['#9467bd', '#8c564b',
        ax.set_yscale('symlog', linthresh=0.1)
        ax.set_title('Custo da agregação por rodada (ms)')

        # 6. Comunicação
        ax = axes[1, 2]
        kb = tabela['Upload por cliente/rodada (KB)']
        ax.bar(kb.index, kb.values, color=[CORES[m] for m in kb.index])
        for i, v in enumerate(kb.values):
            ax.text(i, v, f"{v:,.1f} KB", ha='center', va='bottom')
        ax.set_yscale('log')
        ax.set_title('Upload por cliente por rodada')

        plt.tight_layout()
        plt.show()

    print("\nFim do Teste 5 (FedAvg em claro × SMPC × CKKS)!")
```

```
================================================================================
 BASE: CREDITCARD
================================================================================
Carregando '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/creditcard/c
Treino: (170883, 29) | Validação: (56962, 29) | Teste: (56962, 29) | Fraudes
Amostras normais por cliente: [56863, 56863, 56862]
Autoencoder: 29 features de entrada, 2,219 parâmetros treináveis.

--- FedAvg em claro (setup: 0.00s) ---
  Rodada 1/5 | ROC-AUC val: 0.9424 | erro máx. agregação: 0.00e+00 | agregac
  Rodada 2/5 | ROC-AUC val: 0.9435 | erro máx. agregação: 0.00e+00 | agregac
  Rodada 3/5 | ROC-AUC val: 0.9460 | erro máx. agregação: 0.00e+00 | agregac
  Rodada 4/5 | ROC-AUC val: 0.9491 | erro máx. agregação: 0.00e+00 | agregac
  Rodada 5/5 | ROC-AUC val: 0.9520 | erro máx. agregação: 0.00e+00 | agregac

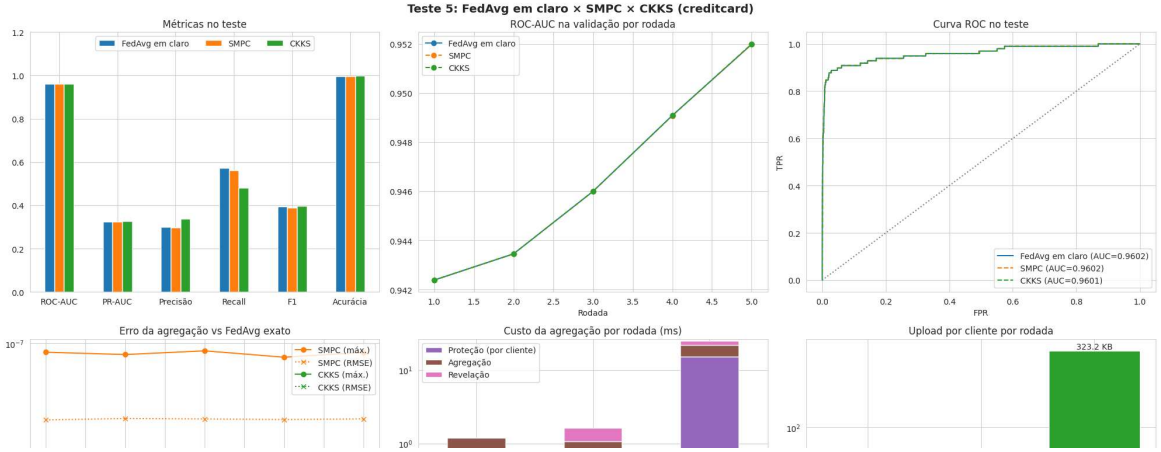
--- SMPC (setup: 0.00s) ---
  Rodada 1/5 | ROC-AUC val: 0.9424 | erro máx. agregação: 8.67e-08 | agregac
  Rodada 2/5 | ROC-AUC val: 0.9435 | erro máx. agregação: 8.36e-08 | agregac
  Rodada 3/5 | ROC-AUC val: 0.9460 | erro máx. agregação: 8.87e-08 | agregac
  Rodada 4/5 | ROC-AUC val: 0.9491 | erro máx. agregação: 8.01e-08 | agregac
  Rodada 5/5 | ROC-AUC val: 0.9520 | erro máx. agregação: 8.65e-08 | agregac

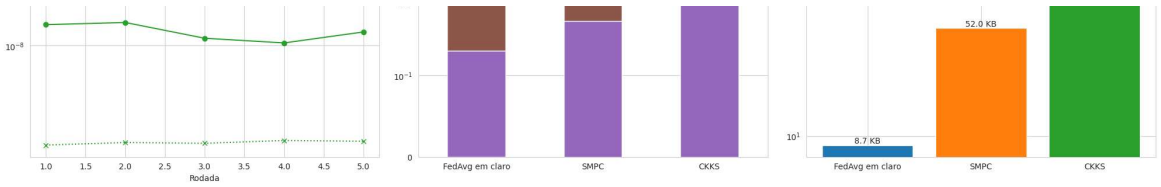
--- CKKS (setup: 0.07s) ---
  Rodada 1/5 | ROC-AUC val: 0.9424 | erro máx. agregação: 1.40e-08 | agregac
  Rodada 2/5 | ROC-AUC val: 0.9435 | erro máx. agregação: 1.44e-08 | agregac
  Rodada 3/5 | ROC-AUC val: 0.9460 | erro máx. agregação: 1.12e-08 | agregac
  Rodada 4/5 | ROC-AUC val: 0.9491 | erro máx. agregação: 1.04e-08 | agregac
  Rodada 5/5 | ROC-AUC val: 0.9520 | erro máx. agregação: 1.24e-08 | agregac

>> MÉTRICAS DE QUALIDADE NO TESTE (limiar escolhido na validação):
                ROC-AUC  PR-AUC Precisão  Recall      F1 Acurácia
Mecanismo
FedAvg em claro  0.9602  0.3251   0.3011  0.5714  0.3944   0.9970
SMPC             0.9602  0.3253   0.2973  0.5612  0.3887   0.9970
CKKS             0.9601  0.3257   0.3381  0.4796  0.3966   0.9975

>> FIDELIDADE E CUSTO DA AGREGAÇÃO SEGURA:
                Erro máx. agregação  Δ máx. pesos finais vs claro  Concordâ
Mecanismo
FedAvg em claro                   0                             0
SMPC                       8.869e-08                      0.008297
CKKS                       1.443e-08                       0.00903

>> COMPARAÇÕES PAREADAS (Δ = mecanismo seguro − claro):
  Claro × SMPC: ΔROC-AUC: +1.14e-05 | ΔPR-AUC: +2.35e-04 | ΔF1: -5.67e-03 |
  Claro × CKKS: ΔROC-AUC: -3.13e-05 | ΔPR-AUC: +6.08e-04 | ΔF1: +2.26e-03 |
```






```
================================================================================
 BASE: PAYSIM
================================================================================
Carregando '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/paysi
Amostragem estratificada de 1,000,000 linhas (preserva a taxa de fraude)...
Treino: (600000, 8) | Validação: (200000, 8) | Teste: (200000, 8) | Fraudes
Amostras normais por cliente: [199407, 199407, 199407]
Autoencoder: 8 features de entrada, 1,190 parâmetros treináveis.

--- FedAvg em claro (setup: 0.00s) ---
  Rodada 1/5 | ROC-AUC val: 0.8656 | erro máx. agregação: 0.00e+00 | agregac
  Rodada 2/5 | ROC-AUC val: 0.8550 | erro máx. agregação: 0.00e+00 | agregac
  Rodada 3/5 | ROC-AUC val: 0.8576 | erro máx. agregação: 0.00e+00 | agregac
  Rodada 4/5 | ROC-AUC val: 0.8562 | erro máx. agregação: 0.00e+00 | agregac
  Rodada 5/5 | ROC-AUC val: 0.8554 | erro máx. agregação: 0.00e+00 | agregac

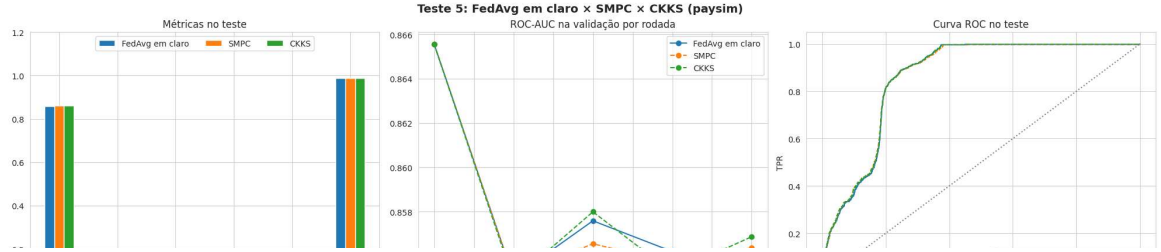
--- SMPC (setup: 0.00s) ---
  Rodada 1/5 | ROC-AUC val: 0.8656 | erro máx. agregação: 8.94e-08 | agregac
  Rodada 2/5 | ROC-AUC val: 0.8550 | erro máx. agregação: 8.32e-08 | agregac
  Rodada 3/5 | ROC-AUC val: 0.8566 | erro máx. agregação: 8.44e-08 | agregac
  Rodada 4/5 | ROC-AUC val: 0.8557 | erro máx. agregação: 8.57e-08 | agregac
  Rodada 5/5 | ROC-AUC val: 0.8564 | erro máx. agregação: 8.01e-08 | agregac

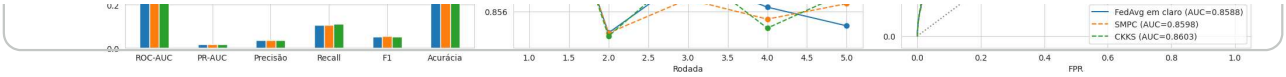
--- CKKS (setup: 0.07s) ---
  Rodada 1/5 | ROC-AUC val: 0.8656 | erro máx. agregação: 1.34e-08 | agregac
  Rodada 2/5 | ROC-AUC val: 0.8549 | erro máx. agregação: 1.25e-08 | agregac
  Rodada 3/5 | ROC-AUC val: 0.8580 | erro máx. agregação: 1.02e-08 | agregac
  Rodada 4/5 | ROC-AUC val: 0.8552 | erro máx. agregação: 1.62e-08 | agregac
  Rodada 5/5 | ROC-AUC val: 0.8569 | erro máx. agregação: 1.14e-08 | agregac

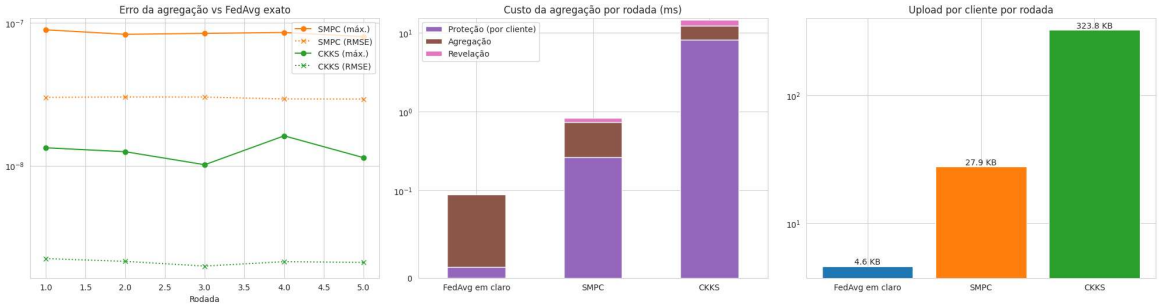
>> MÉTRICAS DE QUALIDADE NO TESTE (limiar escolhido na validação):
                ROC-AUC  PR-AUC Precisão  Recall      F1 Acurácia
Mecanismo
FedAvg em claro  0.8588  0.0173   0.0351  0.1046  0.0526   0.9888
SMPC             0.8598  0.0176   0.0363  0.1046  0.0539   0.9891
CKKS             0.8603  0.0181   0.0347  0.1113  0.0529   0.9882

>> FIDELIDADE E CUSTO DA AGREGAÇÃO SEGURA:
                Erro máx. agregação  Δ máx. pesos finais vs claro  Concordâ
Mecanismo
FedAvg em claro                   0                            0
SMPC                       8.941e-08                       0.2077
CKKS                       1.619e-08                        0.491

>> COMPARAÇÕES PAREADAS (Δ = mecanismo seguro − claro):
  Claro × SMPC: ΔROC-AUC: +9.74e-04 | ΔPR-AUC: +3.28e-04 | ΔF1: +1.28e-03 |
  Claro × CKKS: ΔROC-AUC: +1.49e-03 | ΔPR-AUC: +8.11e-04 | ΔF1: +3.19e-04 |
```








```
Fim do Teste 5 (FedAvg em claro × SMPC × CKKS)!
```

Nova base: Healthcare Fraud Detection

As células abaixo foram **adicionadas** ao Teste 5, sem alterar as anteriores. Elas repetem o experimento da seção 5 (FedAvg em claro × SMPC × CKKS com o mesmo Autoencoder) com a **mesma configuração** — modelo, hiperparâmetros, sementes, split, limiar e métricas — na base `healthcare_fraud_detection.csv`, carregada direto do Google Drive (`/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/`). O rótulo de fraude é a coluna `Is_Fraud`.

1. A primeira célula registra a base (caminho e engenharia de features) no pipeline do teste.
2. A segunda executa o experimento na base healthcare.
3. A terceira compara os resultados com as outras bases executadas neste notebook.

Engenharia de features da base healthcare (a mesma nos seis testes):

- removidos `Provider_ID` e `Claim_ID` (identificadores, como `nameOrig`/`nameDest` na PaySim) e `Claim_Submission_Date` (carimbo de tempo, como `Time` na Credit Card);
- `claim_amount_log` e `approved_amount_log` (log dos valores, como `amount_log`) e `approved_ratio` = valor aprovado ÷ valor solicitado (análoga ao `errorBalance` da PaySim);
- One-Hot das categóricas (`Patient_Gender`, `Diagnosis_Code`, `Procedure_Code`, `Insurance_Type`, `Provider_Specialty`, `Patient_State`, `Claim_Status`, `Visit_Type`), com ausentes na categoria `Desconhecido`;
- mediana para numéricas ausentes (`Prior_Visits_12m`).

```
# ======================================================================
# --- NOVA BASE: Healthcare Fraud Detection (mesma configuração do teste) --
# ======================================================================
CAMINHOS['healthcare'] = [
    '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/healthcare_f
    'base_de_dados/healthcare_fraud_detection.csv',        # Cópia no reposi
    'PoC/base_de_dados/healthcare_fraud_detection.csv',    # Executando da r
]
COLUNAS_CATEGORICAS_HC = ['Patient_Gender', 'Diagnosis_Code', 'Procedure_Cod
                          'Provider_Specialty', 'Patient_State', 'Claim_Stat


def preparar_healthcare(df):
    df = df.dropna(subset=['Is_Fraud'])  # sem amostragem, como a base Credi
    df = df.copy()
    df['Is_Fraud'] = df['Is_Fraud'].astype(int)
    # Identificadores não generalizam (como nameOrig/nameDest); a data é um
```

```
        df = df.drop(columns=['Provider_ID', 'Claim_ID', 'Claim_Submission_Date'
        # Log dos valores (como amount_log) e proporção aprovada pela operadora
        df['approved_ratio'] = df['Approved_Amount'] / df['Claim_Amount'].replac
        df['claim_amount_log'] = np.log1p(df['Claim_Amount'].clip(lower=0))
        df['approved_amount_log'] = np.log1p(df['Approved_Amount'].clip(lower=0)
        df = df.drop(columns=['Claim_Amount', 'Approved_Amount'])
        # Categóricas -> One-Hot (Procedure_Code é código, não quantidade); ause
        for col in COLUNAS_CATEGORICAS_HC:
            df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhec
        df = pd.get_dummies(df, columns=COLUNAS_CATEGORICAS_HC, drop_first=True,
        # Numéricas ausentes (Prior_Visits_12m) -> mediana
        for col in df.select_dtypes(include=np.number).columns:
            if df[col].isnull().sum() > 0:
                df[col] = df[col].fillna(df[col].median())
        return df, 'Is_Fraud'


    PREPARADORES['healthcare'] = preparar_healthcare
    DATASETS_HEALTHCARE = ['healthcare']
    print("Base healthcare registrada:", CAMINHOS['healthcare'][0])
```

```
    Base healthcare registrada: /content/drive/MyDrive/Mestrado Cesar/Base de Dad
```

```
    # ======================================================================
    # --- 5b. EXPERIMENTO NA BASE HEALTHCARE (mesma configuração da seção 5) ---
    # ======================================================================

    MECANISMOS = [AgregadorClaro, AgregadorSMPC, AgregadorCKKS]
    METRICAS = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1', 'Acurácia']
    CORES = {'FedAvg em claro': '#1f77b4', 'SMPC': '#ff7f0e', 'CKKS': '#2ca02c'}
    resultados_gerais = globals().get('resultados_gerais', {})  # preserva os re

    for nome_base in DATASETS_HEALTHCARE:
        print("\n" + "=" * 78)
        print(f" BASE: {nome_base.upper()}")
        print("=" * 78)
        dados = carregar_dados(nome_base)
        if dados is None:
            print(f"Aviso: base '{nome_base}' não encontrada em {CAMINHOS[nome_b
            continue

        # Pesos iniciais idênticos para os três mecanismos
        seed_everything(SEED)
        estado_inicial = copy.deepcopy(TabularAutoencoder(dados['input_dim']).to
        n_params = sum(t.numel() for t in estado_inicial.values())
        print(f"Autoencoder: {dados['input_dim']} features de entrada, {n_params

        execucoes = {}
        for Mecanismo in MECANISMOS:
            agregador = Mecanismo()
            print(f"\n--- {agregador.nome} (setup: {agregador.tempo_setup:.2f}s)
            modelo, historico = executar_fl(agregador, dados, estado_inicial)
            execucoes[agregador.nome] = dict(modelo=modelo, historico=historico,
```

```
                                                avaliacao=avaliar(modelo, dados), t

        # ----- Tabela comparativa -----
        claro = execucoes['FedAvg em claro']
        linhas = []
        for nome_mec, ex in execucoes.items():
            av, hist = ex['avaliacao'], ex['historico']
            overhead_ms = 1000 * (hist['t_protecao'] + hist['t_agregacao'] + his
            linha = {'Mecanismo': nome_mec, **{m: av[m] for m in METRICAS},
                     'Erro máx. agregação': hist['erro_max'].max(),
                     'Δ máx. pesos finais vs claro': (ex['vetor'] - claro['vetor
                     'Concordância pred. vs claro (%)': 100 * np.mean(av['y_pred
                     'Correlação scores vs claro': np.corrcoef(av['scores'], cla
                     'Overhead agregação (ms/rodada)': overhead_ms,
                     'Upload por cliente/rodada (KB)': hist['bytes_cliente'].ilo
                     'Setup (s)': ex['tempo_setup'],
                     'Treino local (s/rodada)': hist['t_treino'].mean()}
            linhas.append(linha)
        tabela = pd.DataFrame(linhas).set_index('Mecanismo')
        resultados_gerais[nome_base] = dict(tabela=tabela, execucoes=execucoes)

        pd.set_option('display.width', 200)
        print("\n>> MÉTRICAS DE QUALIDADE NO TESTE (limiar escolhido na validaçã
        print(tabela[METRICAS].map(lambda x: f"{x:.4f}").to_string())
        print("\n>> FIDELIDADE E CUSTO DA AGREGAÇÃO SEGURA:")
        print(tabela.drop(columns=METRICAS).to_string(float_format=lambda x: f"{

        print("\n>> COMPARAÇÕES PAREADAS (Δ = mecanismo seguro − claro):")
        for seguro in ['SMPC', 'CKKS']:
            t = tabela.loc[seguro]
            deltas = " | ".join(f"Δ{m}: {t[m] - tabela.loc['FedAvg em claro', m]
            fator = t['Overhead agregação (ms/rodada)'] / max(tabela.loc['FedAvg
            banda = t['Upload por cliente/rodada (KB)'] / tabela.loc['FedAvg em
            print(f"  Claro × {seguro}: {deltas} | concordância {t['Concordância
                  f"| overhead {fator:,.0f}x | banda {banda:.1f}x")

        # ----- Visualizações -----
        fig, axes = plt.subplots(2, 3, figsize=(20, 11))
        fig.suptitle(f"Teste 5: FedAvg em claro × SMPC × CKKS ({nome_base})", fo

        # 1. Métricas de qualidade
        ax = axes[0, 0]
        tabela[METRICAS].T.plot.bar(ax=ax, color=[CORES[m] for m in tabela.index
        ax.set_title('Métricas no teste')
        ax.set_ylim(0, 1.2)
        ax.legend(loc='upper center', ncol=3)

        # 2. Convergência
        ax = axes[0, 1]
        for nome_mec, ex in execucoes.items():
            estilo = '-' if nome_mec == 'FedAvg em claro' else '--'
            ax.plot(ex['historico']['rodada'], ex['historico']['roc_auc_val'], e
                    color=CORES[nome_mec], label=nome_mec)
        ax.set_title('ROC-AUC na validação por rodada')
        ax.set_xlabel('Rodada')
```

```
        ax.legend()

        # 3. Curvas ROC no teste
        ax = axes[0, 2]
        for nome_mec, ex in execucoes.items():
            fpr, tpr, _ = roc_curve(dados['y_test'], ex['avaliacao']['scores'])
            estilo = '-' if nome_mec == 'FedAvg em claro' else '--'
            ax.plot(fpr, tpr, estilo, color=CORES[nome_mec], label=f"{nome_mec}
        ax.plot([0, 1], [0, 1], color='gray', linestyle=':')
        ax.set_title('Curva ROC no teste')
        ax.set_xlabel('FPR')
        ax.set_ylabel('TPR')
        ax.legend(loc='lower right')

        # 4. Erro numérico da agregação
        ax = axes[1, 0]
        for nome_mec in ['SMPC', 'CKKS']:
            h = execucoes[nome_mec]['historico']
            ax.plot(h['rodada'], h['erro_max'], marker='o', color=CORES[nome_mec
            ax.plot(h['rodada'], h['erro_rmse'], marker='x', linestyle=':', colo
        ax.set_yscale('log')
        ax.set_title('Erro da agregação vs FedAvg exato')
        ax.set_xlabel('Rodada')
        ax.legend()

        # 5. Tempo de agregação por etapa
        ax = axes[1, 1]
        tempos = pd.DataFrame({nome_mec: 1000 * ex['historico'][['t_protecao', '
                               for nome_mec, ex in execucoes.items()}).T
        tempos.columns = ['Proteção (por cliente)', 'Agregação', 'Revelação']
        tempos.plot.bar(stacked=True, ax=ax, rot=0, color=['#9467bd', '#8c564b',
        ax.set_yscale('symlog', linthresh=0.1)
        ax.set_title('Custo da agregação por rodada (ms)')

        # 6. Comunicação
        ax = axes[1, 2]
        kb = tabela['Upload por cliente/rodada (KB)']
        ax.bar(kb.index, kb.values, color=[CORES[m] for m in kb.index])
        for i, v in enumerate(kb.values):
            ax.text(i, v, f"{v:,.1f} KB", ha='center', va='bottom')
        ax.set_yscale('log')
        ax.set_title('Upload por cliente por rodada')

        plt.tight_layout()
        plt.show()

    print("\nFim do Teste 5 (FedAvg em claro × SMPC × CKKS) na base healthcare!"
```

```
================================================================================
 BASE: HEALTHCARE
================================================================================
Carregando '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/healt
Treino: (6000, 48) | Validação: (2000, 48) | Teste: (2000, 48) | Fraudes no
Amostras normais por cliente: [1835, 1834, 1834]
Autoencoder: 48 features de entrada, 3,150 parâmetros treináveis.

--- FedAvg em claro (setup: 0.00s) ---
  Rodada 1/5 | ROC-AUC val: 0.8978 | erro máx. agregação: 0.00e+00 | agrega
  Rodada 2/5 | ROC-AUC val: 0.8994 | erro máx. agregação: 0.00e+00 | agrega
  Rodada 3/5 | ROC-AUC val: 0.9026 | erro máx. agregação: 0.00e+00 | agrega
  Rodada 4/5 | ROC-AUC val: 0.9057 | erro máx. agregação: 0.00e+00 | agrega
  Rodada 5/5 | ROC-AUC val: 0.9081 | erro máx. agregação: 0.00e+00 | agrega

--- SMPC (setup: 0.00s) ---
  Rodada 1/5 | ROC-AUC val: 0.8978 | erro máx. agregação: 8.64e-08 | agrega
  Rodada 2/5 | ROC-AUC val: 0.8994 | erro máx. agregação: 8.37e-08 | agrega
  Rodada 3/5 | ROC-AUC val: 0.9026 | erro máx. agregação: 8.42e-08 | agrega
  Rodada 4/5 | ROC-AUC val: 0.9057 | erro máx. agregação: 8.33e-08 | agrega
  Rodada 5/5 | ROC-AUC val: 0.9081 | erro máx. agregação: 8.40e-08 | agrega

--- CKKS (setup: 0.07s) ---
  Rodada 1/5 | ROC-AUC val: 0.8978 | erro máx. agregação: 1.36e-08 | agrega
  Rodada 2/5 | ROC-AUC val: 0.8994 | erro máx. agregação: 1.12e-08 | agrega
  Rodada 3/5 | ROC-AUC val: 0.9026 | erro máx. agregação: 1.25e-08 | agrega
  Rodada 4/5 | ROC-AUC val: 0.9057 | erro máx. agregação: 1.72e-08 | agrega
  Rodada 5/5 | ROC-AUC val: 0.9081 | erro máx. agregação: 1.40e-08 | agrega

>> MÉTRICAS DE QUALIDADE NO TESTE (limiar escolhido na validação):
               ROC-AUC  PR-AUC Precisão  Recall      F1 Acurácia
```

```
# ================================================================
# --- COMPARATIVO ENTRE AS BASES (Teste 5: FedAvg em claro × SMPC × CKKS) --
# ================================================================
import contextlib
import io

NOMES_BASES = {'creditcard': 'Credit Card', 'paysim': 'PaySim', 'healthcare'
# Cor fixa por base (paleta categórica validada para daltonismo): a mesma ba
CORES_BASES = {'creditcard': '#2a78d6', 'healthcare': '#eb6834', 'paysim': '
bases = [b for b in NOMES_BASES if b in globals().get('resultados_gerais', {


def perfil_da_base(nome):
    """Tamanho e taxa de fraude do conjunto de teste (a base é recarregada s
    with contextlib.redirect_stdout(io.StringIO()):
        d = carregar_dados(nome)
    return {'Transações (teste)': len(d['y_test']), 'Taxa de fraude (teste)'
            'Features': int(d['input_dim'])}

ORDEM_MECANISMOS = ['FedAvg em claro', 'SMPC', 'CKKS']

if not bases:
    print("Nenhum resultado disponível: execute a seção 5 e/ou a base health
else:
    perfis = {b: perfil_da_base(b) for b in bases}
```

```
registros = []
for b in bases:
    for mecanismo, linha in resultados_gerais[b]['tabela'].iterrows():
        registros.append({'Base': NOMES_BASES[b], 'Mecanismo': mecanismo
longo = pd.DataFrame(registros)

print(">> CARACTERÍSTICAS DAS BASES")
for b in bases:
    p = perfis[b]
    print(f"  {NOMES_BASES[b]:12s} teste: {p['Transações (teste)']:,} tr
          f"| {p['Features']} features")

print("\n>> MÉTRICAS NO TESTE POR MECANISMO E BASE")
for m in ['ROC-AUC', 'PR-AUC', 'F1']:
    print(f"\n{m}:")
    print(longo.pivot(index='Mecanismo', columns='Base', values=m).reind

taxa_por_base = {NOMES_BASES[b]: perfis[b]['Taxa de fraude (teste)'] for
longo['PR-AUC normalizado'] = ((longo['PR-AUC'] - longo['Base'].map(taxa
                               / (1 - longo['Base'].map(taxa_por_base)))
print("\nPR-AUC normalizado = (PR-AUC − taxa de fraude) / (1 − taxa):")
print(longo.pivot(index='Mecanismo', columns='Base', values='PR-AUC norm
      .round(4).to_string())

print("\n>> EFEITO DA AGREGAÇÃO SEGURA (mecanismo seguro − FedAvg em cla
for b in bases:
    t = resultados_gerais[b]['tabela']
    for seguro in ['SMPC', 'CKKS']:
        deltas = " | ".join(f"Δ{m} {t.loc[seguro, m] - t.loc['FedAvg em
        print(f"  {NOMES_BASES[b]:12s} Claro × {seguro}: {deltas} | conc
              f"{t.loc[seguro, 'Concordância pred. vs claro (%)']:.3f}%"

print("\n>> CUSTO DA AGREGAÇÃO POR BASE (cresce com o nº de parâmetros,
for coluna in ['Overhead agregação (ms/rodada)', 'Upload por cliente/rod
    print(f"\n{coluna}:")
    print(longo.pivot(index='Mecanismo', columns='Base', values=coluna).
          .to_string(float_format=lambda v: f"{v:.4g}"))
print("\nObservação: o ROC-AUC não depende da taxa de fraude e é a métri
      "O PR-AUC depende dela (um classificador aleatório tem PR-AUC igua
      "do PR-AUC entre bases deve usar o PR-AUC normalizado = (PR-AUC −

fig, axes = plt.subplots(1, 3, figsize=(22, 5.5))
largura = 0.8 / len(bases)
x = np.arange(len(ORDEM_MECANISMOS))
for eixo, (coluna, titulo) in zip(axes, [('ROC-AUC', 'ROC-AUC no teste')
                                         ('Overhead agregação (ms/rodada
    for i, b in enumerate(bases):
        valores = resultados_gerais[b]['tabela'].reindex(ORDEM_MECANISMO
        eixo.bar(x + (i - (len(bases) - 1) / 2) * largura, valores, larg
    eixo.set_xticks(x)
    eixo.set_xticklabels(ORDEM_MECANISMOS)
    eixo.set_title(titulo, fontweight='bold')
    eixo.legend(loc='upper center', ncol=len(bases), frameon=False)
axes[0].set_ylim(0, 1.2)
axes[1].set_ylim(0, 1.2)
```

```
        axes[2].set_yscale('log')
        axes[2].set_ylim(top=axes[2].get_ylim()[1] * 10)  # folga no topo para a
        plt.suptitle('Teste 5 — FedAvg em claro × SMPC × CKKS: comparativo entre
        plt.tight_layout()
        plt.show()
```

```
>> CARACTERÍSTICAS DAS BASES
  Credit Card  teste: 56,962 transações | taxa de fraude 0.17% | 29 features
  PaySim       teste: 200,000 transações | taxa de fraude 0.30% | 8 features
  Healthcare   teste: 2,000 transações | taxa de fraude 8.30% | 48 features

>> MÉTRICAS NO TESTE POR MECANISMO E BASE

ROC-AUC:
Base             Credit Card  Healthcare  PaySim
Mecanismo
FedAvg em claro       0.9602      0.9092  0.8588
SMPC                  0.9602      0.9092  0.8598
CKKS                  0.9601      0.9092  0.8603

PR-AUC:
Base             Credit Card  Healthcare  PaySim
Mecanismo
FedAvg em claro       0.3251      0.5099  0.0173
SMPC                  0.3253      0.5099  0.0176
CKKS                  0.3257      0.5099  0.0181

F1:
Base             Credit Card  Healthcare  PaySim
Mecanismo
FedAvg em claro       0.3944      0.5728  0.0526
SMPC                  0.3887      0.5728  0.0539
CKKS                  0.3966      0.5728  0.0529

PR-AUC normalizado = (PR-AUC − taxa de fraude) / (1 − taxa):
Base             Credit Card  Healthcare  PaySim
Mecanismo
FedAvg em claro       0.3299      0.4655  0.0144
SMPC                  0.3242      0.4655  0.0147
CKKS                  0.3246      0.4655  0.0152

>> EFEITO DA AGREGAÇÃO SOBRE FedAvg em claro (SMPC e CKKS são iguais)
  Credit Card  Claro × SMPC: ΔROC-AUC +1.1e-05 | ΔPR-AUC +2.4e-04 | ΔF1 -5.7
  Credit Card  Claro × CKKS: ΔROC-AUC -3.1e-05 | ΔPR-AUC +6.1e-04 | ΔF1 +2.3
  PaySim       Claro × SMPC: ΔROC-AUC +9.7e-04 | ΔPR-AUC +3.3e-04 | ΔF1 +1.3
  PaySim       Claro × CKKS: ΔROC-AUC +1.5e-03 | ΔPR-AUC +8.1e-04 | ΔF1 +3.2
  Healthcare   Claro × SMPC: ΔROC-AUC +0.0e+00 | ΔPR-AUC +0.0e+00 | ΔF1 +0.0
  Healthcare   Claro × CKKS: ΔROC-AUC +0.0e+00 | ΔPR-AUC +0.0e+00 | ΔF1 +0.0

>> AGREGAÇÃO POR BASE (cresce com o nº de parâmetros, que depende d
Overhead, agregação (
Mecanismo
SMPC                1.638         1.14  0.8294
CKKS                   25        15.34    14.6
Upload por cliente/ro
Base             Credit Card  Healthcare  PaySim
Mecanismo
FedAvg em claro       2.668        10.3   1.648
SMPC                  [illegible]        [illegible]  27.89
CKKS                  333.2        323.4   323.8
Erro máx. agregação:
```

Novas bases: Bank Account Fraud (NeurIPS 2022), Insurance Claims e Fraud Oracle

As células abaixo foram **adicionadas** ao Teste 5, sem alterar as anteriores. Elas repetem o experimento da seção 5 (FedAvg em claro × SMPC × CKKS com o mesmo Autoencoder) com a **mesma configuração** — modelo, hiperparâmetros, sementes, split, limiar e métricas — em mais três bases.

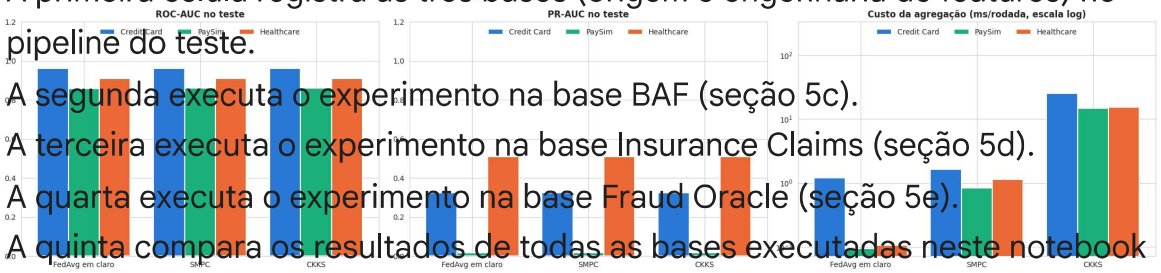
Origem das bases:

- **Bank Account Fraud Dataset Suite (NeurIPS 2022)** — só o arquivo `Base.csv` (213 MB, 1 milhão de solicitações de abertura de conta, ~1,1% de fraude), baixado com `kagglehub` (já instalado no Colab; localmente, `pip install kagglehub`): `kagglehub.dataset_download("sgpjesus/bank-account-fraud-dataset-neurips-2022", path="Base.csv")`. Sem acesso ao Kaggle, é usada a cópia do repositório `PoC/base_de_dados/Base.csv.zip`.
- **Insurance Claims** — `insurance_claims.csv` (1.000 sinistros de seguro de automóvel, 24,7% de fraude), no repositório GitHub do projeto, em `PoC/base_de_dados/insurance_claims.csv`; a célula usa a cópia local quando o notebook roda a partir do repositório e, no Colab, lê o arquivo direto do GitHub.

- **Fraud Oracle** — `fraud_oracle.csv` (15.420 sinistros de seguro de veículos, 6,0% de fraude; rótulo `FraudFound_P`: 1 = fraude, 0 = não fraude), no repositório GitHub do projeto, em `PoC/base_de_dados/fraud_oracle.csv`, lida da mesma forma que a Insurance Claims.

Base Mecanismo PaySim
FedAvg em claro, 0
SMPC
CKKS 1.443e-08 1.722e-08 1.619e-08

Observação: o ROC-AUC não depende da taxa de fraude e é a métrica mais direta



1. A primeira célula registra as três bases (origem e engenharia de features) no pipeline do teste.
2. A segunda executa o experimento na base BAF (seção 5c).
3. A terceira executa o experimento na base Insurance Claims (seção 5d).
4. A quarta executa o experimento na base Fraud Oracle (seção 5e).
5. A quinta compara os resultados de todas as bases executadas neste notebook (Credit Card, Healthcare, BAF, Insurance Claims e Fraud Oracle).

Tamanho das bases: vale a mesma regra da PaySim neste teste (`PAYSIM_AMOSTRA` = 1 milhão de linhas, amostra estratificada acima disso). A BAF tem exatamente 1 milhão de linhas, a Insurance Claims 1.000 e a Fraud Oracle 15.420; as três entram inteiras.

Engenharia de features (a mesma nos seis testes):

- **BAF** (rótulo `fraud_bool`): removidos `month` (mês da solicitação, carimbo de tempo como `Time` na Credit Card) e `device_fraud_count` (constante na `Base.csv`); nas seis colunas em que valor negativo significa ausente (`prev_address_months_count`, `current_address_months_count`, `intended_balcon_amount`, `bank_months_count`, `session_length_in_minutes`, `device_distinct_emails_8w`), criado um indicador `_ausente` e o valor ausente vai para a mediana; log de `intended_balcon_amount`, `days_since_request`, `zip_count_4w` e `proposed_credit_limit`; One-Hot de `payment_type`, `employment_status`, `housing_status`, `source` e `device_os`.
- **Insurance Claims** (rótulo `fraud_reported`, Y/N → 1/0): removidos `policy_number`, `insured_zip` e `incident_location` (identificadores, como `nameOrig` / `nameDest` na PaySim), `_c39` (coluna vazia) e `auto_model` (39 modelos para 1.000 linhas; `auto_make` é mantida); as datas `policy_bind_date` e `incident_date` viram `dias_apolice_ate_incidente`; log de `total_claim_amount`, `injury_claim`, `property_claim` e `vehicle_claim`; One-Hot das 16 categóricas, com `?` na categoria `Desconhecido` e `authorities_contacted` ausente (texto `None` no CSV) como `Nenhuma`.
- **Fraud Oracle** (rótulo `FraudFound_P`): removidos `PolicyNumber` e `RepNumber` (identificadores), `Year` (carimbo de tempo, como `Time`) e `PolicyType` (repete `VehicleCategory` + `BasePolicy`); `Month` e

`MonthClaimed` viram `meses_ate_sinistro`; as nove colunas de faixas em texto (`VehiclePrice`, `Days_Policy_Accident`, `Days_Policy_Claim`, `PastNumberOfClaims`, `AgeOfVehicle`, `AgeOfPolicyHolder`, `NumberOfSuppliments`, `AddressChange_Claim`, `NumberOfCars`) viram números em ordem crescente; `Age` = 0 (inválido) vai para a mediana; One-Hot das 12 categóricas, com `0` em `DayOfWeekClaimed` como `Desconhecido`.

```
# ======================================================================
# --- NOVAS BASES: Bank Account Fraud (Base.csv), Insurance Claims e Fraud O
# ======================================================================


def baixar_base_baf():
    """Baixa só o arquivo Base.csv com kagglehub (None se o Kaggle não estiv
    try:
        import kagglehub
        # Download latest version — apenas o arquivo Base.csv (213 MB)
        path = kagglehub.dataset_download("sgpjesus/bank-account-fraud-datas
        print("Path to dataset files:", path)
        return path
    except Exception as erro:
        print(f"Download pelo kagglehub indisponível ({type(erro).__name__})
        return None


CAMINHOS['baf'] = [c for c in [
    baixar_base_baf(),
    'base_de_dados/Base.csv.zip',      # Cópia no repositório (zip lido dire
    'PoC/base_de_dados/Base.csv.zip',  # Executando da raiz do repositório
    'base_de_dados/Base.csv',
    'PoC/base_de_dados/Base.csv',
] if c]
# Insurance Claims: arquivo no repositório GitHub do projeto (PoC/base_de_da
CAMINHOS['insurance'] = [
    'base_de_dados/insurance_claims.csv',        # Executando de dentro de P
    'PoC/base_de_dados/insurance_claims.csv',    # Executando da raiz do rep
]
if not any(os.path.exists(c) for c in CAMINHOS['insurance']):
    # No Colab (sem o repositório clonado), baixa o arquivo direto do GitHub
    import urllib.request
    try:
        urllib.request.urlretrieve('https://raw.githubusercontent.com/suxlcm
        CAMINHOS['insurance'].append('insurance_claims.csv')
        print("insurance_claims.csv baixado do repositório GitHub.")
    except Exception as erro:
        print(f"Não foi possível baixar insurance_claims.csv do GitHub ({typ
# Fraud Oracle: arquivo no repositório GitHub do projeto (PoC/base_de_dados/
CAMINHOS['oracle'] = [
    'base_de_dados/fraud_oracle.csv',        # Executando de dentro de PoC/
    'PoC/base_de_dados/fraud_oracle.csv',    # Executando da raiz do reposit
]
if not any(os.path.exists(c) for c in CAMINHOS['oracle']):
```

```
        # No Colab (sem o repositório clonado), baixa o arquivo direto do GitHub
        import urllib.request
        try:
            urllib.request.urlretrieve('https://raw.githubusercontent.com/suxlcm
            CAMINHOS['oracle'].append('fraud_oracle.csv')
            print("fraud_oracle.csv baixado do repositório GitHub.")
        except Exception as erro:
            print(f"Não foi possível baixar fraud_oracle.csv do GitHub ({type(er


    def reduzir(df, alvo):
        """Mesma regra de tamanho da PaySim neste teste: acima de PAYSIM_AMOSTRA
        df = df.dropna(subset=[alvo])
        if PAYSIM_AMOSTRA and len(df) > PAYSIM_AMOSTRA:
            print(f"Amostragem estratificada de {PAYSIM_AMOSTRA:,} linhas (prese
            df, _ = train_test_split(df, train_size=PAYSIM_AMOSTRA, stratify=df[
        return df.copy()


    def preparar_baf(df):
        df = reduzir(df, 'fraud_bool')
        df['fraud_bool'] = df['fraud_bool'].astype(int)
        # month: carimbo de tempo (como Time); device_fraud_count: constante (ze
        df = df.drop(columns=['month', 'device_fraud_count'], errors='ignore')
        # Valor negativo nestas colunas codifica "ausente" (documentação do data
        for col in ['prev_address_months_count', 'current_address_months_count',
                    'bank_months_count', 'session_length_in_minutes', 'device_di
            df[col + '_ausente'] = (df[col] < 0).astype(float)
            df[col] = df[col].where(df[col] >= 0)
        # Contagens e valores com cauda longa -> log (como amount_log nas outras
        for col in ['intended_balcon_amount', 'days_since_request', 'zip_count_4
            df[col + '_log'] = np.log1p(df[col].clip(lower=0))
        df = df.drop(columns=['intended_balcon_amount', 'days_since_request', 'z
        categoricas = ['payment_type', 'employment_status', 'housing_status', 's
        for col in categoricas:
            df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhec
        df = pd.get_dummies(df, columns=categoricas, drop_first=True, dtype=floa
        for col in df.select_dtypes(include=np.number).columns:
            if df[col].isnull().sum() > 0:
                df[col] = df[col].fillna(df[col].median())
        return df, 'fraud_bool'


    def preparar_insurance(df):
        df = df.copy()
        df['fraud_reported'] = df['fraud_reported'].astype(str).str.strip().str.
        df = reduzir(df, 'fraud_reported')
        df['fraud_reported'] = df['fraud_reported'].astype(int)
        # Datas -> um único intervalo (dias entre a contratação da apólice e o i
        df['dias_apolice_ate_incidente'] = (pd.to_datetime(df['incident_date'],
                                            - pd.to_datetime(df['policy_bind_dat
        # Identificadores (como nameOrig/nameDest), coluna vazia, auto_model (39
        df = df.drop(columns=['policy_number', 'insured_zip', 'incident_location
                              'policy_bind_date', 'incident_date'], errors='igno
        # Valores do sinistro com cauda longa -> log (como amount_log nas outras
```

```
        for col in ['total_claim_amount', 'injury_claim', 'property_claim', 'veh
            df[col + '_log'] = np.log1p(df[col].clip(lower=0))
        df = df.drop(columns=['total_claim_amount', 'injury_claim', 'property_cl
        # '?' = desconhecido; authorities_contacted 'None' (lido como ausente) =
        df['authorities_contacted'] = df['authorities_contacted'].fillna('Nenhum
        categoricas = ['policy_state', 'policy_csl', 'insured_sex', 'insured_edu
                       'insured_hobbies', 'insured_relationship', 'incident_type
                       'authorities_contacted', 'incident_state', 'incident_city
                       'police_report_available', 'auto_make']
        for col in categoricas:
            df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhec
        df = pd.get_dummies(df, columns=categoricas, drop_first=True, dtype=floa
        for col in df.select_dtypes(include=np.number).columns:
            if df[col].isnull().sum() > 0:
                df[col] = df[col].fillna(df[col].median())
        return df, 'fraud_reported'


    def preparar_oracle(df):
        df = reduzir(df, 'FraudFound_P')
        df['FraudFound_P'] = df['FraudFound_P'].astype(int)
        # PolicyNumber (sequencial) e RepNumber (representante) são identificado
        # Year é carimbo de tempo, como 'Time' na base Credit Card; PolicyType r
        # Month e MonthClaimed viram um único intervalo: meses entre o acidente
        MESES = {'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4, 'May': 5, 'Jun': 6, 'Ju
        df['meses_ate_sinistro'] = (df['MonthClaimed'].map(MESES) - df['Month'].
        colunas_remover = ['PolicyNumber', 'RepNumber', 'Year', 'PolicyType', 'M
        df = df.drop(columns=[col for col in colunas_remover if col in df.column
        # Faixas de texto -> números em ordem crescente (preserva a ordem e evit
        FAIXAS_ORDINAIS = {
            'VehiclePrice': ['less than 20000', '20000 to 29000', '30000 to 3900
            'Days_Policy_Accident': ['none', '1 to 7', '8 to 15', '15 to 30', 'm
            'Days_Policy_Claim': ['none', '8 to 15', '15 to 30', 'more than 30']
            'PastNumberOfClaims': ['none', '1', '2 to 4', 'more than 4'],
            'AgeOfVehicle': ['new', '2 years', '3 years', '4 years', '5 years',
            'AgeOfPolicyHolder': ['16 to 17', '18 to 20', '21 to 25', '26 to 30'
            'NumberOfSuppliments': ['none', '1 to 2', '3 to 5', 'more than 5'],
            'AddressChange_Claim': ['under 6 months', '1 year', '2 to 3 years',
            'NumberOfCars': ['1 vehicle', '2 vehicles', '3 to 4', '5 to 8', 'mor
        }
        for col, ordem in FAIXAS_ORDINAIS.items():
            df[col] = df[col].astype(str).str.strip().map({faixa: i for i, faixa
        # Age = 0 é um valor inválido na base (o motorista mais novo está na fai
        df['Age'] = df['Age'].where(df['Age'] > 0)

        # Categóricas -> One-Hot ('0' em DayOfWeekClaimed é valor desconhecido)
        colunas_categoricas = ['DayOfWeek', 'DayOfWeekClaimed', 'Make', 'Acciden
                               'VehicleCategory', 'PoliceReportFiled', 'WitnessP
        for col in colunas_categoricas:
            df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhec
        df = pd.get_dummies(df, columns=colunas_categoricas, drop_first=True, dt

        # Numéricas ausentes (faixas fora da lista, Age = 0, mês desconhecido) -
        for col in df.select_dtypes(include=np.number).columns:
            if df[col].isnull().sum() > 0:
```

```
            df[col] = df[col].fillna(df[col].median())
    return df, 'FraudFound_P'


PREPARADORES['baf'] = preparar_baf
PREPARADORES['insurance'] = preparar_insurance
PREPARADORES['oracle'] = preparar_oracle
DATASETS_BAF = ['baf']
DATASETS_INSURANCE = ['insurance']
DATASETS_ORACLE = ['oracle']
for nome_base in ['baf', 'insurance', 'oracle']:
    print(f"Base {nome_base} registrada; caminhos procurados: {CAMINHOS[nome
```

```
Downloading to /root/.cache/kagglehub/datasets/sgpjesus/bank-account-fraud-da
100%|██████████| 82.4M/82.4M [00:00<00:00, 114MB/s]Extracting zip of Base.csv

Path to dataset files: /root/.cache/kagglehub/datasets/sgpjesus/bank-account-
insurance_claims.csv baixado do repositório GitHub.
fraud_oracle.csv baixado do repositório GitHub.
Base baf registrada; caminhos procurados: ['/root/.cache/kagglehub/datasets/s
Base insurance registrada; caminhos procurados: ['base_de_dados/insurance_cla
Base oracle registrada; caminhos procurados: ['base_de_dados/fraud_oracle.csv
```

```
# ======================================================================
# --- 5c. EXPERIMENTO NA BASE BAF (mesma configuração da seção 5) ---
# ======================================================================

MECANISMOS = [AgregadorClaro, AgregadorSMPC, AgregadorCKKS]
METRICAS = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1', 'Acurácia']
CORES = {'FedAvg em claro': '#1f77b4', 'SMPC': '#ff7f0e', 'CKKS': '#2ca02c'}
resultados_gerais = globals().get('resultados_gerais', {})  # preserva os re

for nome_base in DATASETS_BAF:
    print("\n" + "=" * 78)
    print(f" BASE: {nome_base.upper()}")
    print("=" * 78)
    dados = carregar_dados(nome_base)
    if dados is None:
        print(f"Aviso: base '{nome_base}' não encontrada em {CAMINHOS[nome_b
        continue

    # Pesos iniciais idênticos para os três mecanismos
    seed_everything(SEED)
    estado_inicial = copy.deepcopy(TabularAutoencoder(dados['input_dim']).to
    n_params = sum(t.numel() for t in estado_inicial.values())
    print(f"Autoencoder: {dados['input_dim']} features de entrada, {n_params

    execucoes = {}
    for Mecanismo in MECANISMOS:
        agregador = Mecanismo()
        print(f"\n--- {agregador.nome} (setup: {agregador.tempo_setup:.2f}s)
        modelo, historico = executar_fl(agregador, dados, estado_inicial)
        execucoes[agregador.nome] = dict(modelo=modelo, historico=historico,
```

```
                                                avaliacao=avaliar(modelo, dados), t

        # ----- Tabela comparativa -----
        claro = execucoes['FedAvg em claro']
        linhas = []
        for nome_mec, ex in execucoes.items():
            av, hist = ex['avaliacao'], ex['historico']
            overhead_ms = 1000 * (hist['t_protecao'] + hist['t_agregacao'] + his
            linha = {'Mecanismo': nome_mec, **{m: av[m] for m in METRICAS},
                     'Erro máx. agregação': hist['erro_max'].max(),
                     'Δ máx. pesos finais vs claro': (ex['vetor'] - claro['vetor
                     'Concordância pred. vs claro (%)': 100 * np.mean(av['y_pred
                     'Correlação scores vs claro': np.corrcoef(av['scores'], cla
                     'Overhead agregação (ms/rodada)': overhead_ms,
                     'Upload por cliente/rodada (KB)': hist['bytes_cliente'].ilo
                     'Setup (s)': ex['tempo_setup'],
                     'Treino local (s/rodada)': hist['t_treino'].mean()}
            linhas.append(linha)
        tabela = pd.DataFrame(linhas).set_index('Mecanismo')
        resultados_gerais[nome_base] = dict(tabela=tabela, execucoes=execucoes)

        pd.set_option('display.width', 200)
        print("\n>> MÉTRICAS DE QUALIDADE NO TESTE (limiar escolhido na validaçã
        print(tabela[METRICAS].map(lambda x: f"{x:.4f}").to_string())
        print("\n>> FIDELIDADE E CUSTO DA AGREGAÇÃO SEGURA:")
        print(tabela.drop(columns=METRICAS).to_string(float_format=lambda x: f"{

        print("\n>> COMPARAÇÕES PAREADAS (Δ = mecanismo seguro − claro):")
        for seguro in ['SMPC', 'CKKS']:
            t = tabela.loc[seguro]
            deltas = " | ".join(f"Δ{m}: {t[m] - tabela.loc['FedAvg em claro', m]
            fator = t['Overhead agregação (ms/rodada)'] / max(tabela.loc['FedAvg
            banda = t['Upload por cliente/rodada (KB)'] / tabela.loc['FedAvg em
            print(f"  Claro × {seguro}: {deltas} | concordância {t['Concordância
                  f"| overhead {fator:,.0f}x | banda {banda:.1f}x")

        # ----- Visualizações -----
        fig, axes = plt.subplots(2, 3, figsize=(20, 11))
        fig.suptitle(f"Teste 5: FedAvg em claro × SMPC × CKKS ({nome_base})", fo

        # 1. Métricas de qualidade
        ax = axes[0, 0]
        tabela[METRICAS].T.plot.bar(ax=ax, color=[CORES[m] for m in tabela.index
        ax.set_title('Métricas no teste')
        ax.set_ylim(0, 1.2)
        ax.legend(loc='upper center', ncol=3)

        # 2. Convergência
        ax = axes[0, 1]
        for nome_mec, ex in execucoes.items():
            estilo = '-' if nome_mec == 'FedAvg em claro' else '--'
            ax.plot(ex['historico']['rodada'], ex['historico']['roc_auc_val'], e
                    color=CORES[nome_mec], label=nome_mec)
        ax.set_title('ROC-AUC na validação por rodada')
        ax.set_xlabel('Rodada')
```

```
        ax.legend()

        # 3. Curvas ROC no teste
        ax = axes[0, 2]
        for nome_mec, ex in execucoes.items():
            fpr, tpr, _ = roc_curve(dados['y_test'], ex['avaliacao']['scores'])
            estilo = '-' if nome_mec == 'FedAvg em claro' else '--'
            ax.plot(fpr, tpr, estilo, color=CORES[nome_mec], label=f"{nome_mec}
        ax.plot([0, 1], [0, 1], color='gray', linestyle=':')
        ax.set_title('Curva ROC no teste')
        ax.set_xlabel('FPR')
        ax.set_ylabel('TPR')
        ax.legend(loc='lower right')

        # 4. Erro numérico da agregação
        ax = axes[1, 0]
        for nome_mec in ['SMPC', 'CKKS']:
            h = execucoes[nome_mec]['historico']
            ax.plot(h['rodada'], h['erro_max'], marker='o', color=CORES[nome_mec
            ax.plot(h['rodada'], h['erro_rmse'], marker='x', linestyle=':', colo
        ax.set_yscale('log')
        ax.set_title('Erro da agregação vs FedAvg exato')
        ax.set_xlabel('Rodada')
        ax.legend()

        # 5. Tempo de agregação por etapa
        ax = axes[1, 1]
        tempos = pd.DataFrame({nome_mec: 1000 * ex['historico'][['t_protecao', '
                               for nome_mec, ex in execucoes.items()}).T
        tempos.columns = ['Proteção (por cliente)', 'Agregação', 'Revelação']
        tempos.plot.bar(stacked=True, ax=ax, rot=0, color=['#9467bd', '#8c564b',
        ax.set_yscale('symlog', linthresh=0.1)
        ax.set_title('Custo da agregação por rodada (ms)')

        # 6. Comunicação
        ax = axes[1, 2]
        kb = tabela['Upload por cliente/rodada (KB)']
        ax.bar(kb.index, kb.values, color=[CORES[m] for m in kb.index])
        for i, v in enumerate(kb.values):
            ax.text(i, v, f"{v:,.1f} KB", ha='center', va='bottom')
        ax.set_yscale('log')
        ax.set_title('Upload por cliente por rodada')

        plt.tight_layout()
        plt.show()

    print("\nFim do Teste 5 (FedAvg em claro × SMPC × CKKS) na base BAF!")
```

```
================================================================================
 BASE: BAF
================================================================================
Carregando '/root/.cache/kagglehub/datasets/sgpjesus/bank-account-fraud-data
Treino: (600000, 51) | Validação: (200000, 51) | Teste: (200000, 51) | Fraud
Amostras normais por cliente: [197795, 197794, 197794]
Autoencoder: 51 features de entrada, 3,297 parâmetros treináveis.

--- FedAvg em claro (setup: 0.00s) ---
  Rodada 1/5 | ROC-AUC val: 0.4939 | erro máx. agregação: 0.00e+00 | agrega
  Rodada 2/5 | ROC-AUC val: 0.5512 | erro máx. agregação: 0.00e+00 | agrega
  Rodada 3/5 | ROC-AUC val: 0.5584 | erro máx. agregação: 0.00e+00 | agrega
  Rodada 4/5 | ROC-AUC val: 0.5518 | erro máx. agregação: 0.00e+00 | agrega
  Rodada 5/5 | ROC-AUC val: 0.5447 | erro máx. agregação: 0.00e+00 | agrega

--- SMPC (setup: 0.00s) ---
  Rodada 1/5 | ROC-AUC val: 0.4939 | erro máx. agregação: 8.17e-08 | agrega
  Rodada 2/5 | ROC-AUC val: 0.5513 | erro máx. agregação: 8.46e-08 | agrega
  Rodada 3/5 | ROC-AUC val: 0.5588 | erro máx. agregação: 8.64e-08 | agrega
  Rodada 4/5 | ROC-AUC val: 0.5508 | erro máx. agregação: 8.39e-08 | agrega
  Rodada 5/5 | ROC-AUC val: 0.5451 | erro máx. agregação: 8.43e-08 | agrega

--- CKKS (setup: 0.07s) ---
  Rodada 1/5 | ROC-AUC val: 0.4939 | erro máx. agregação: 1.70e-08 | agrega
  Rodada 2/5 | ROC-AUC val: 0.5510 | erro máx. agregação: 1.73e-08 | agrega
  Rodada 3/5 | ROC-AUC val: 0.5582 | erro máx. agregação: 1.11e-08 | agrega
  Rodada 4/5 | ROC-AUC val: 0.5509 | erro máx. agregação: 1.13e-08 | agrega
  Rodada 5/5 | ROC-AUC val: 0.5447 | erro máx. agregação: 1.14e-08 | agrega

>> MÉTRICAS DE QUALIDADE NO TESTE (limiar escolhido na validação):
                ROC-AUC  PR-AUC Precisão  Recall      F1 Acurácia
```

```
# ================================================================
# --- 5d. EXPERIMENTO NA BASE INSURANCE CLAIMS (mesma configuração da seção
# ================================================================

MECANISMOS = [AgregadorClaro, AgregadorSMPC, AgregadorCKKS]
METRICAS = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1', 'Acurácia']
CORES = {'FedAvg em claro': '#1f77b4', 'SMPC': '#ff7f0e', 'CKKS': '#2ca02c'}
resultados_gerais = globals().get('resultados_gerais', {})  # preserva os re

for nome_base in DATASETS_INSURANCE:
    print("\n" + "=" * 78)
    print(f" BASE: {nome_base.upper()}")
    print("=" * 78)
    dados = carregar_dados(nome_base)
    if dados is None:
        print(f"Aviso: base '{nome_base}' não encontrada em {CAMINHOS[nome_b
        continue

    # Pesos iniciais idênticos para os três mecanismos
    seed_everything(SEED)
    estado_inicial = copy.deepcopy(TabularAutoencoder(dados['input_dim']).to
    n_params = sum(t.numel() for t in estado_inicial.values())
    print(f"Autoencoder: {dados['input_dim']} features de entrada, {n_params

    execucoes = {}
```

```
for Mecanismo in MECANISMOS:
    agregador = Mecanismo()
    print(f"\n--- {agregador.nome} (setup: {agregador.tempo_setup:.2f}s)
    modelo, historico = executar_fl(agregador, dados, estado_inicial)
    execucoes[agregador.nome] = dict(modelo=modelo, historico=historico,
                                     avaliacao=avaliar(modelo, dados), t

# ----- Tabela comparativa -----
claro = execucoes['FedAvg em claro']
linhas = []
for nome_mec, ex in execucoes.items():
    av, hist = ex['avaliacao'], ex['historico']
    overhead_ms = 1000 * (hist['t_protecao'] + hist['t_agregacao'] + his
    linha = {'Mecanismo': nome_mec, **{m: av[m] for m in METRICAS},
             'Erro máx. agregação': hist['erro_max'].max(),
             'Δ máx. pesos finais vs claro': (ex['vetor'] - claro['vetor
             'Concordância pred. vs claro (%)': 100 * np.mean(av['y_pred
             'Correlação scores vs claro': np.corrcoef(av['scores'], cla
             'Overhead agregação (ms/rodada)': overhead_ms,
             'Upload por cliente/rodada (KB)': hist['bytes_cliente'].ilo
             'Setup (s)': ex['tempo_setup'],
             'Treino local (s/rodada)': hist['t_treino'].mean()}
    linhas.append(linha)
tabela = pd.DataFrame(linhas).set_index('Mecanismo')
resultados_gerais[nome_base] = dict(tabela=tabela, execucoes=execucoes)

pd.set_option('display.width', 200)
print("\n>> MÉTRICAS DE QUALIDADE NO TESTE (limiar escolhido na validaçã
print(tabela[METRICAS].map(lambda x: f"{x:.4f}").to_string())
print("\n>> FIDELIDADE E CUSTO DA AGREGAÇÃO SEGURA:")
print(tabela.drop(columns=METRICAS).to_string(float_format=lambda x: f"{

print("\n>> COMPARAÇÕES PAREADAS (Δ = mecanismo seguro − claro):")
for seguro in ['SMPC', 'CKKS']:
    t = tabela.loc[seguro]
    deltas = " | ".join(f"Δ{m}: {t[m] - tabela.loc['FedAvg em claro', m]
    fator = t['Overhead agregação (ms/rodada)'] / max(tabela.loc['FedAvg
    banda = t['Upload por cliente/rodada (KB)'] / tabela.loc['FedAvg em
    print(f"  Claro × {seguro}: {deltas} | concordância {t['Concordância
          f"| overhead {fator:,.0f}x | banda {banda:.1f}x")

# ----- Visualizações -----
fig, axes = plt.subplots(2, 3, figsize=(20, 11))
fig.suptitle(f"Teste 5: FedAvg em claro × SMPC × CKKS ({nome_base})", fo

# 1. Métricas de qualidade
ax = axes[0, 0]
tabela[METRICAS].T.plot.bar(ax=ax, color=[CORES[m] for m in tabela.index
ax.set_title('Métricas no teste')
ax.set_ylim(0, 1.2)
ax.legend(loc='upper center', ncol=3)

# 2. Convergência
ax = axes[0, 1]
for nome_mec, ex in execucoes.items():
```

```
            estilo = '-' if nome_mec == 'FedAvg em claro' else '--'
            ax.plot(ex['historico']['rodada'], ex['historico']['roc_auc_val'], e
                    color=CORES[nome_mec], label=nome_mec)
        ax.set_title('ROC-AUC na validação por rodada')
        ax.set_xlabel('Rodada')
        ax.legend()

        # 3. Curvas ROC no teste
        ax = axes[0, 2]
        for nome_mec, ex in execucoes.items():
            fpr, tpr, _ = roc_curve(dados['y_test'], ex['avaliacao']['scores'])
            estilo = '-' if nome_mec == 'FedAvg em claro' else '--'
            ax.plot(fpr, tpr, estilo, color=CORES[nome_mec], label=f"{nome_mec}
        ax.plot([0, 1], [0, 1], color='gray', linestyle=':')
        ax.set_title('Curva ROC no teste')
        ax.set_xlabel('FPR')
        ax.set_ylabel('TPR')
        ax.legend(loc='lower right')

        # 4. Erro numérico da agregação
        ax = axes[1, 0]
        for nome_mec in ['SMPC', 'CKKS']:
            h = execucoes[nome_mec]['historico']
            ax.plot(h['rodada'], h['erro_max'], marker='o', color=CORES[nome_mec
            ax.plot(h['rodada'], h['erro_rmse'], marker='x', linestyle=':', colo
        ax.set_yscale('log')
        ax.set_title('Erro da agregação vs FedAvg exato')
        ax.set_xlabel('Rodada')
        ax.legend()

        # 5. Tempo de agregação por etapa
        ax = axes[1, 1]
        tempos = pd.DataFrame({nome_mec: 1000 * ex['historico'][['t_protecao', '
                               for nome_mec, ex in execucoes.items()}).T
        tempos.columns = ['Proteção (por cliente)', 'Agregação', 'Revelação']
        tempos.plot.bar(stacked=True, ax=ax, rot=0, color=['#9467bd', '#8c564b',
        ax.set_yscale('symlog', linthresh=0.1)
        ax.set_title('Custo da agregação por rodada (ms)')

        # 6. Comunicação
        ax = axes[1, 2]
        kb = tabela['Upload por cliente/rodada (KB)']
        ax.bar(kb.index, kb.values, color=[CORES[m] for m in kb.index])
        for i, v in enumerate(kb.values):
            ax.text(i, v, f"{v:,.1f} KB", ha='center', va='bottom')
        ax.set_yscale('log')
        ax.set_title('Upload por cliente por rodada')

        plt.tight_layout()
        plt.show()

    print("\nFim do Teste 5 (FedAvg em claro × SMPC × CKKS) na base Insurance Cl
```